# Customer Enquiry Tracker

Build a small web app that receives customer enquiries, stores them, categorizes them automatically and helps a business track follow-up.

### Time and submission window

Spend approximately **3 hours**. Submit within **72 hours of receiving the assignment email**; the email will specify the exact deadline in PKT. Stop at the time limit and document unfinished work. A clear, working core is more valuable than extra features.

### Submit an enquiry

Create a public form for name, email and message. Validate on both client and server. Show loading, success and error feedback. Reject blank fields and malformed email addresses.

### Save before categorizing

Persist the enquiry first, with a unique ID, creation time and status New. Then attempt categorization. If saving fails, show an error and do not report success. If AI fails, retain the enquiry.

### Categorize automatically

Call the supplied mock AI service from the backend. Accept only Sales, Support, Billing or Other. Track Pending, Completed or Failed separately from the business status. Offer manual retry for Failed records.

### Review and update

Build a simple business dashboard showing enquiries newest first. Allow full-message viewing, filtering by status, status changes, and retrying failed categorization. Changes must survive page refresh.

### Allowed stack and boundaries

Use React or Next.js, Node.js or Python, and any persistent database. SQLite is acceptable. Next.js server routes are acceptable; separate frontend/backend deployments are unnecessary. Local execution is sufficient.

**Not required:** login, roles, email or WhatsApp notifications, n8n, background queues, charts, payments, a real LLM API, or deployment. Use synthetic customer data only. Treat the dashboard as a local assessment demo.

# Data and integration contract

### Minimum enquiry record

**id** (unique), **name**, **email**, **message**, **createdAt**, **status**, **category** (nullable), **categorizationState**.

Status: New / In Progress / Resolved.

Categorization state: Pending / Completed / Failed.

Category stays null until a valid categorization succeeds. Other is a valid category, not an error fallback.

### Suggested API surface

POST /api/enquiries - validate, save, attempt categorization.

GET /api/enquiries?status=New - list newest first; filter optional.

PATCH /api/enquiries/:id - update status.

POST /api/enquiries/:id/retry - retry failed categorization.

Equivalent routes are acceptable. Document request/response examples and return suitable HTTP errors for invalid input, missing records and server failures.

### Supplied mock AI service

Requires Node.js 18+. Run **node mock-ai-service.mjs**.

Default endpoint: **POST http://127.0.0.1:8787/categorize**.

JSON request: {"enquiryId":"unique-id","message":"customer text"}

Success response: {"category":"Sales"}. Configure the service URL on the backend, not in browser code. No API key, account or installation is needed.

### Repeatable test messages

**Success:** "Please send pricing for your automation service." returns Sales.

**Temporary failure:** "[FAIL_ONCE] I have an invoice question." returns HTTP 503 on its first attempt, then Billing on retry.

**Invalid AI output:** "[INVALID_ONCE] My login is broken." first returns category Unexpected, then Support on retry.

Attempts are tracked per enquiryId in mock-service memory. Restarting the service resets them. Supply the saved enquiry ID on every attempt.

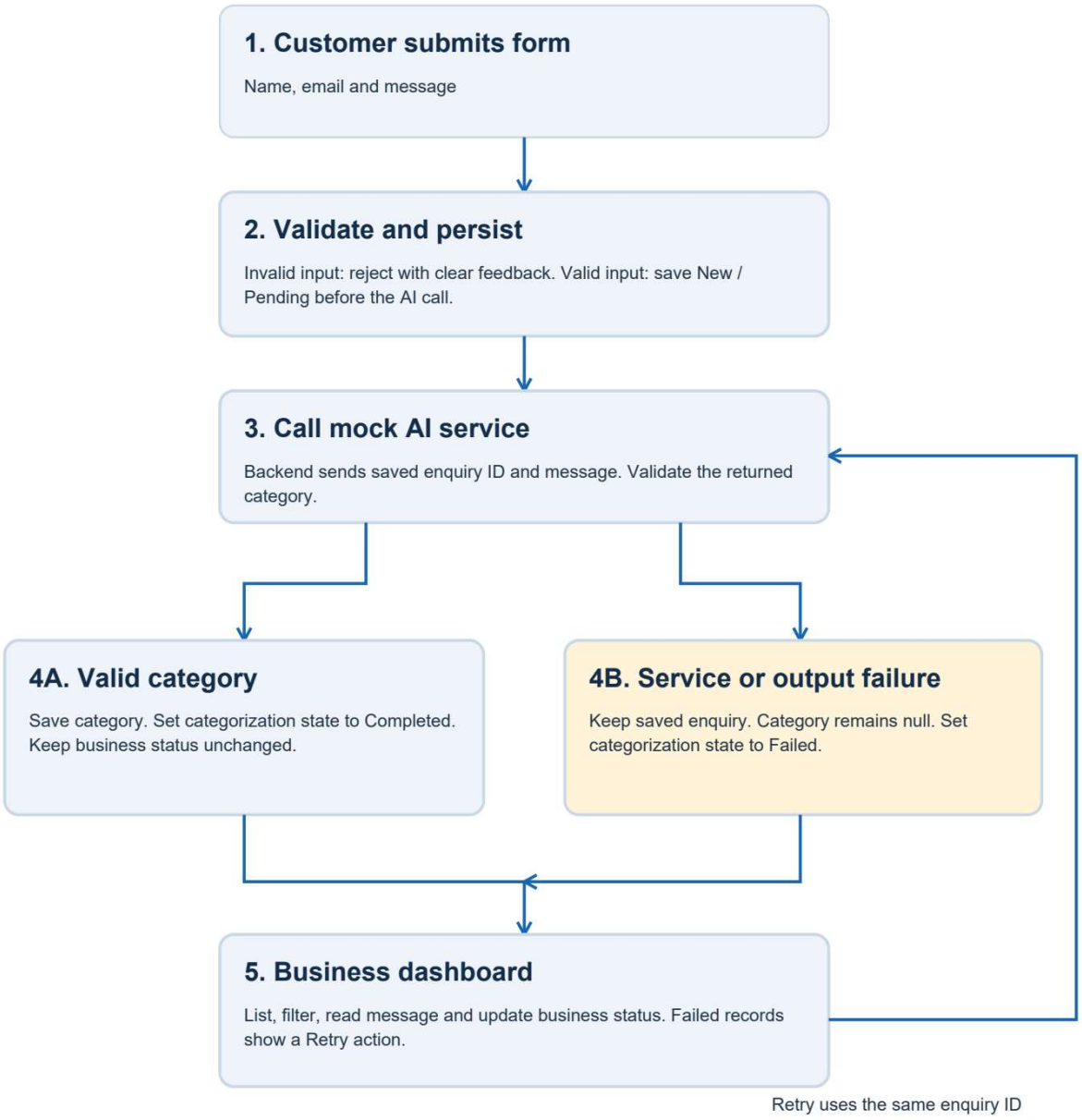
### Implementation choices to explain

Synchronous categorization is acceptable. Use a bounded request timeout. Save Failed on timeout, service error or invalid category. Do not create another enquiry when retrying. A successful retry updates the existing record. Avoid exposing internal errors or secrets to the user.

If your environment cannot run the supplied mock, implement an in-process adapter with the same request fields and three test behaviours. Explain the substitution in your README. A real LLM integration earns no extra credit.

# Process flow

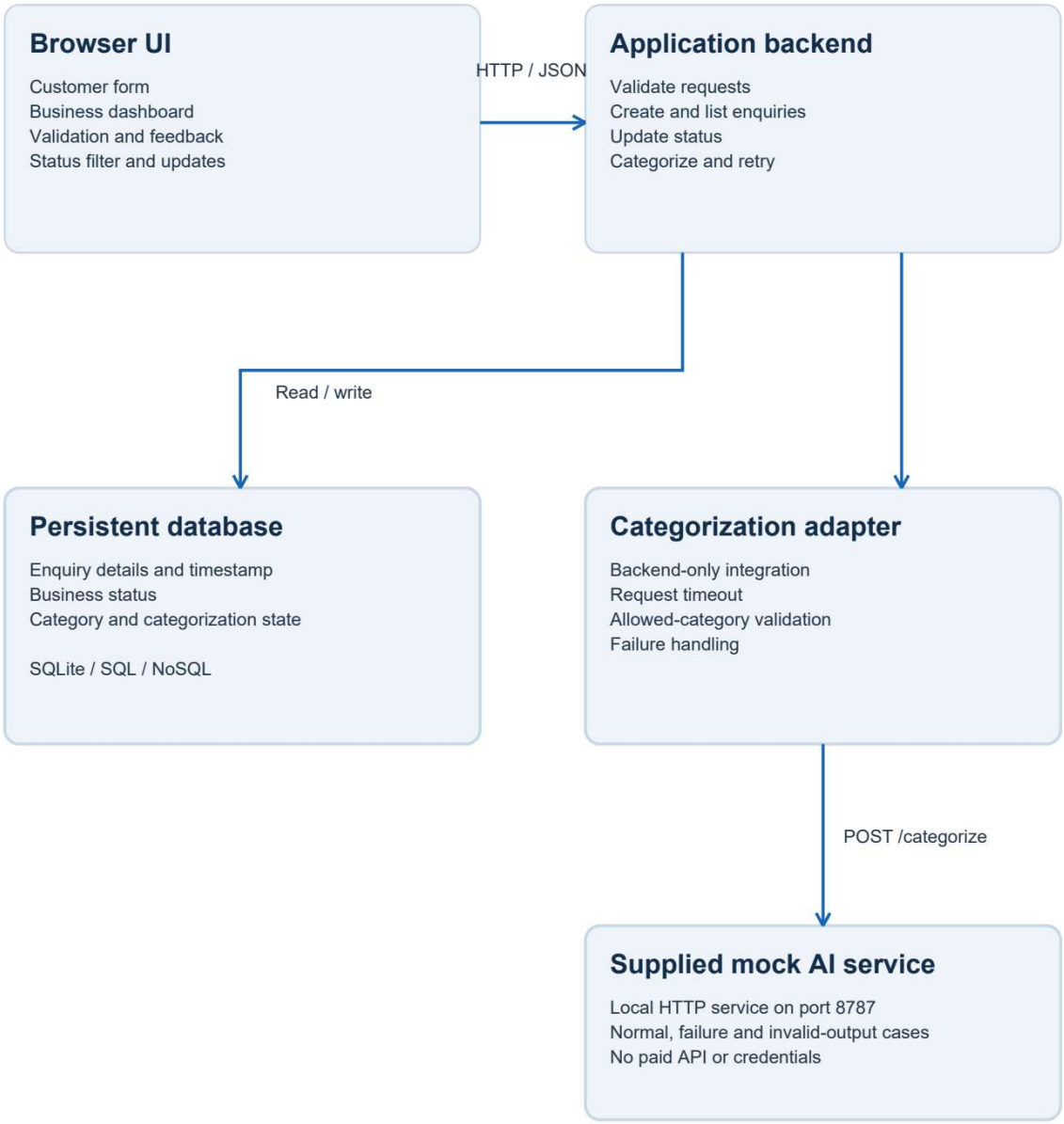
Required behaviour. The diagram shows outcomes; choose your own code structure.




**Important:** a categorization failure must not lose the customer enquiry or appear as a successful category. A refresh must preserve saved data and status changes.

# Reference architecture

A logical design, not a required framework layout. A single Next.js app may contain both UI and server routes.




**Explain your own design:** show your folder structure, the form-to-database path, the categorization boundary and how the existing record is updated after a retry. Include your own small diagram in the README; it may differ from this reference.

# Submission and assessment

### Submit these items

1. Repository link or ZIP containing source code, dependency manifest and lockfile where applicable.
2. README with exact setup commands, database initialization and mock-service instructions.
3. Example environment file without secrets.
4. Your own brief architecture/process diagram and request/response examples.
5. Verification notes with results for the checks below.
6. Time spent, limitations, tradeoffs and AI-tool disclosure.

### Verification checklist

A. Valid enquiry saves and survives refresh.

B. Empty fields and malformed email are rejected by the server.

C. Valid category is stored and shown.

D. [FAIL_ONCE] preserves the record and shows Failed.

E. Retry recovers without creating a duplicate enquiry.

F. [INVALID_ONCE] is rejected, then recovers on retry.

G. Status changes survive refresh.

H. Status filter returns the correct enquiries.

A manual checklist is acceptable. Meaningful automated tests are welcome but are not mandatory within the time limit.

### How we will assess the work

**35%** Working functionality and persistence.

**25%** Understanding demonstrated in the walkthrough.

**20%** Code clarity and separation of responsibilities.

**10%** Validation and error handling.

**10%** Setup instructions, verification and explanation.

Extra features, paid integrations and elaborate styling receive no additional points. Partial submissions can still show useful reasoning.

### AI assistance and ownership

AI coding tools are allowed. State which tools you used, what they helped produce, and how you verified or corrected the output. Submit work you can explain. Do not include secrets, real customer data or code you do not have permission to share.

### 15 minute follow up walkthrough

Demonstrate one successful submission and one failed categorization with retry. Explain one frontend component, backend route and database operation. Discuss one tradeoff and one improvement. Be ready to explain how you would add priority or replace the mock with a real LLM; no additional feature build is required.